

AI Analytics System
ner Support Dataset

AUTHOR Majd Massalha

POSITION AI Specialist Engineer Assignment — DriveNets

DATE May 28, 2026

MODEL Claude Sonnet 4.6

 Demo Video — available upon request

 Rep

232
PASSING TESTS

26,872
DATASET ROWS

4
ROUTE T

01 Executive Summary

CORE RESULT

I built a production-oriented customer-service analytics agent over the Bitext dataset (26,872 rows). The system answers structured analytical questions using deterministic DuckDB tools, routes queries through a keyword-based LangGraph router, exposes tools through FastMCP, and includes secure persistent memory, operational traces, and an evaluation/governance harness with **232 passing tests** and zero LLM calls on the analytical path.

26,872

DATASET ROWS

232

PASSING TESTS

5

DETERMINISTIC
TOOLS

4

ROUTE TYPES

What was built

- **Structured analytics** — exact counts, distributions, and category breakdowns via DuckDB
- **Unstructured summarization** — extractive examples from dataset rows via LLM formatting
- **Out-of-scope refusal** — explicit rejection of non-domain queries with a redirecting message
- **Multi-step reasoning** — LangGraph StateGraph with 4-way conditional routing
- **Persistent memory** — short-term session checkpoints + long-term user profile store
- **MCP tool exposure** — FastMCP server wrapping all analytics tools with Pydantic validation
- **CLI reasoning traces** — operational trace output: route, confidence, tool, observation, answer

Outcome

A fully tested, security-reviewed, and evaluated agent that never lets the LLM calculate statistics, never concatenates raw user input into SQL, and never exposes chain-of-thought to the operational trace. All numerical answers are enforced from DuckDB. The architecture is modular, reproducible, and ready for a production harness extension.

02

Problem Statement

Customer service operations generate large volumes of structured interaction data. Business stakeholders need fast, accurate answers to questions like:

- *"How many ORDER requests did we receive this month?"*
- *"What are the top 5 complaint categories?"*
- *"Show me examples of refund-related queries."*

CORE FAILURE MODE IN NAIVE LLM SYSTEMS

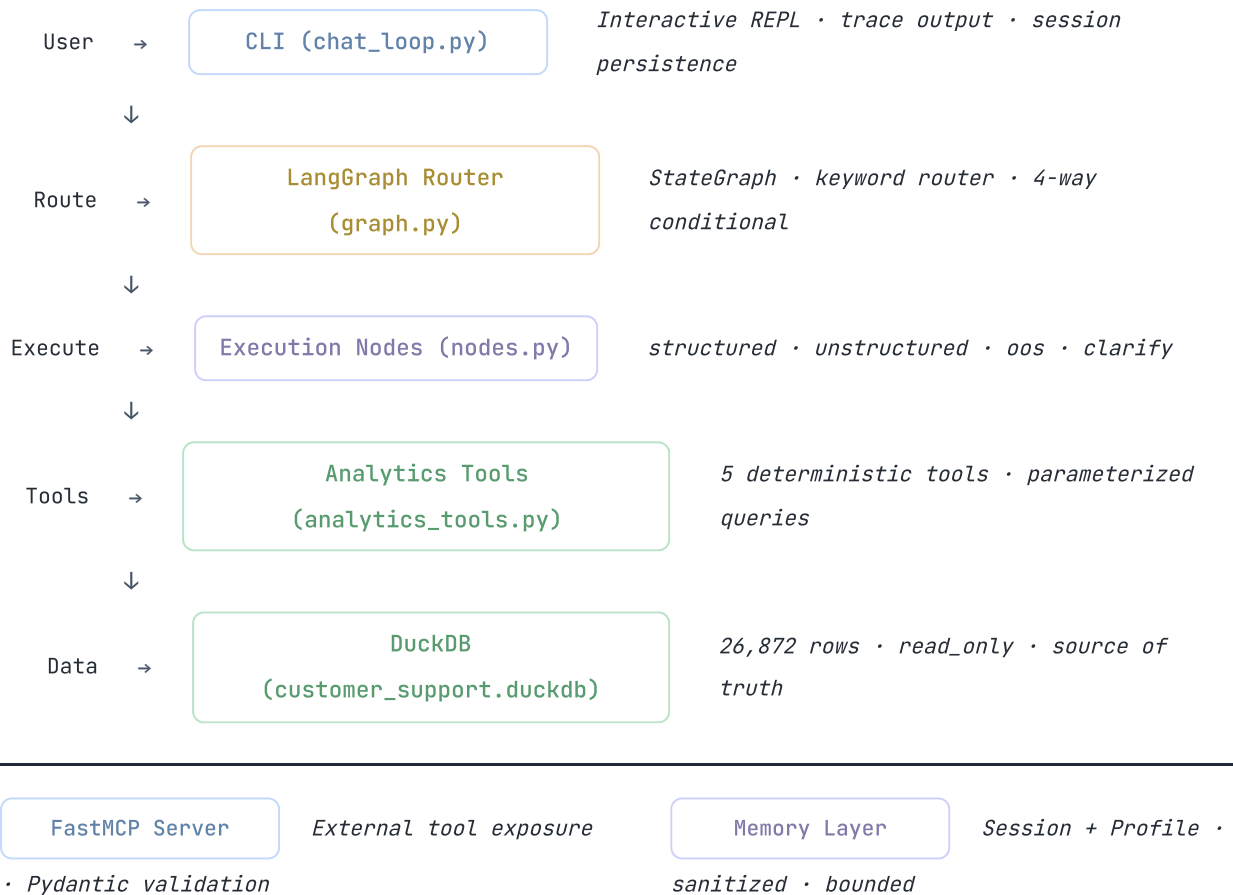
A vanilla LLM-based chatbot will **hallucinate numerical answers** when asked about dataset statistics — inventing counts, distributions, and percentages that do not match the actual data. This is a critical reliability failure for any analytics use case.

Requirements derived from the problem

REQUIREMENT	RATIONALE	IMPLEMENTATION DECISION
Numerical answers must be exact	LLM counts are unreliable	DuckDB as single source of truth
SQL queries must be safe	User-derived input in queries → injection	Parameterized queries + input validation
Out-of-scope must be refused	LLM will answer anything without a boundary	Explicit OOS route with redirecting message
Tools must be externally accessible	Multi-agent ecosystems require MCP	FastMCP server with Pydantic schemas
Memory must be bounded	Long-term memory can leak sensitive data	NEVER_STORE frozenset + 500-char limit
LLM used for routing and formatting only	Keeps analytical path deterministic	Keyword router; LLM for text generation only

03 Architecture & Approach

SYSTEM ARCHITECTURE — REQUEST FLOW



Layer responsibilities

DuckDB — Source of Truth

All numerical answers come from here. 26,872 rows from the Bitext dataset. Read-only connection. Parameterized queries only — no string concatenation.

analytics_tools.py — Deterministic Layer

5 tools: `count_total_rows`, `count_by_category`, `count_by_intent`, `get_top_categories`, `search_examples`. Each validates inputs and logs execution time.

FastMCP — External Exposure

Thin Pydantic validation wrapper around the analytics tools. Independently startable server. No business logic duplication. MCP-compatible for multi-agent ecosystems.

LangGraph Router — Orchestration

`START → route_query → {4 branches} → END`. Keyword-based deterministic router with no LLM calls at routing time. ~0.001s routing

latency. Confidence score returned with each decision.

Memory Layer — Persistence

Two-tier: short-term session checkpoints (last query, route, tool) + long-term user profile (preferred topics, categories). A `_NEVER_STORE` frozenset blocks sensitive keys. Observations never stored.

CLI — Runtime

Interactive REPL with `EXIT_COMMANDS = {exit, quit, q}`. Operational trace printed per turn: route, confidence, tool, truncated observation, final answer. Chain-of-thought never exposed.

Key architectural decisions

- **Keyword router, not LLM router:** eliminates API latency, unpredictability, and cost on the routing path.
 - **Separate tool validation from business logic:** MCP schemas validate inputs; tools focus on SQL execution.
 - **Session ≠ Profile:** short-term metadata (last route, last tool) vs. long-term preferences (preferred categories) are stored separately to limit blast radius if one corrupts.
 - **LLM only for text generation:** formatting summaries and refusal messages. Never for counting, filtering, or routing.
-

04 Development Process

The project was developed iteratively through **Claude Code CLI**. Each phase was prompted with a scoped, constrained instruction — and then reviewed through `git diff`, manual CLI testing, and `pytest` before proceeding. Below are the six representative prompts that shaped the architecture.

PROMPT 01

Dataset Ingestion & Schema Inspection

WHAT I ASKED

Build deterministic dataset ingestion only. Load the Bitext customer support dataset from Hugging Face, save it to DuckDB, and print a schema inspection report. No tools, no routing, no LLM calls.

WHY

I needed to verify that the dataset loaded correctly and that column names, row counts, and category distribution were what I expected before building anything on top of it. Starting with data validation is a non-negotiable step in analytics systems.

WHAT CLAUDE PRODUCED

`src/data/ingest.py` — 87 lines. Loads the Bitext dataset via Hugging Face `datasets` library, converts to pandas, saves to DuckDB as `customer_support_train` table with 26,872 rows. Logs table name and row count on completion.

HOW I VERIFIED

Ran `python -c "from src.data.ingest import load_dataset; load_dataset()"` and checked `outputs/agent.log` for the row count confirmation. Opened the DuckDB file and inspected the schema manually.

WHAT I CHANGED

Added `read_only=True` to DuckDB connections opened later in analytics tools — Claude's initial ingest connection was read-write, but subsequent consumers should never mutate the database.

WHAT I ASKED

Implement 5 analytics tools with parameterized DuckDB queries only. Tools: `count_total_rows`, `count_by_category`, `count_by_intent`, `get_top_categories`, `search_examples`. Validate all inputs. Never concatenate user input into SQL. Log execution time for each call.

WHY

These tools are the entire analytical foundation of the system. Any SQL injection vulnerability here would be a critical security flaw. Parameterization is not optional.

WHAT CLAUDE PRODUCED

`src/tools/analytics_tools.py` — 148 lines. All 5 tools with `_require_nonempty_str()` and `_require_positive_int()` validators. DuckDB `contains()` used for substring search (treats wildcards as literals, eliminating SQL wildcard injection).

WHAT I CHANGED

Added explicit boolean exclusion to `_require_nonempty_str()` — Python's `bool` inherits from `int`, so `isinstance(True, str)` is False but `isinstance(True, int)` is True. Without the check, `True` would pass the positive-int validator.

PROMPT 03

FastMCP Tool Exposure

WHAT I ASKED

Expose all 5 analytics tools through FastMCP. Validate inputs with Pydantic schemas in a separate `schemas.py` file. The MCP server must be a thin wrapper — no business logic duplication. It must be independently startable without the CLI.

WHAT CLAUDE PRODUCED

`src/mcp/server.py` (92 lines) and `src/mcp/schemas.py` (56 lines). Pydantic models for all 4 parameterized tools. Server delegates directly to `analytics_tools` functions.

WHAT I CHANGED

Removed a redundant inline validation check Claude had added inside the MCP tool function that duplicated the Pydantic schema validation. The Pydantic layer is the single validation point — duplication creates inconsistency risk.

WHAT I ASKED

Build a deterministic keyword-based LangGraph router. Route types: structured, unstructured, out_of_scope, needs_clarification. No LLM calls at routing time. Return a `RouteResult` with route, confidence (0.5-0.95), reason_summary, next_action. Priority: 00S first, then clarification, then structured, then unstructured.

WHY

LLM-based routing adds API latency, introduces non-determinism, and costs money per query. A keyword router adds zero latency and is fully testable.

WHAT CLAUDE PRODUCED

`src/agent/router.py` (234 lines) with ~60 keyword signals, regex pattern matching, and confidence levels. `src/agent/graph.py` (58 lines) with the LangGraph `StateGraph` and conditional edge fan-out.

WHAT I CHANGED

Discovered a routing bug during manual validation (see Section 06). Refined tool-selection logic in `nodes.py` to map `count_by_category("ORDER")` correctly for category-specific count queries. Added regression tests before committing the fix.

PROMPT 05**Memory Layer — Session + Profile****WHAT I ASKED**

Build two-tier memory: short-term session checkpoints and long-term user profile. Session stores last query (500 char limit), route, tool, category. Profile stores preferred_topics, frequent_categories. Never store observations, raw dataset rows, API keys, passwords, or credentials. Sanitize session IDs against path traversal.

WHAT CLAUDE PRODUCED

`src/memory/checkpoints.py` (68 lines) and `src/memory/user_profile.py` (81 lines). `_NEVER_STORE` frozenset blocks 10 sensitive key patterns. Session ID sanitization with `re.sub()` + 64-char cap.

WHAT I CHANGED

Added a defensive strip on profile save to prevent accidental whitespace-based leakage. Also added graceful fallback on corrupted JSON — Claude's initial version raised an unhandled `JSONDecodeError` on corrupted session files.

WHAT I ASKED

Build the full evaluation and security harness: golden dataset evaluation plan (33 cases across 6 categories), security audit document, adversarial test cases (SQL injection, prompt injection, unicode, jailbreak), and failure mode tests (corrupted files, missing DB, edge inputs). All tests must be deterministic with no external dependencies.

WHAT CLAUDE PRODUCED

`docs/evaluation_plan.md` (174 lines with 33-entry golden dataset), `docs/security_audit.md`, `tests/test_adversarial_queries.py` (249 lines), `tests/test_failure_modes.py` (267 lines), `tests/test_security_edges.py` (327 lines). Total test count: 232.

WHAT I VERIFIED

Ran the full test suite. Verified that all adversarial payloads (e.g., `' ; DROP TABLE--`, `UNION SELECT`, Unicode RTL override) returned zero rows or raised validation errors — never caused query execution or data leakage.

05 Iterations, Refinements & Decision Points

Bug discovered during manual CLI validation

ROUTING BUG — CATEGORY-SPECIFIC COUNT QUERIES

Symptom: The query *"How many ORDER requests?"* was incorrectly routed to `count_total_rows()` (returning 26,872) instead of `count_by_category("ORDER")` (returning the correct category count).

Root cause: The structured execution node (`nodes.py`) used only "how many" signal detection without checking whether the query also contained a specific category name. The tool-selection logic defaulted to `count_total_rows` for any "how many" query.

RESOLUTION STEPS

1. Reproduced the bug manually with `python main.py --session debug`
2. Identified the tool-selection logic in `src/agent/nodes.py` as the fault location
3. Refined the logic: check for category-specific keywords first; fall back to `count_total_rows` only when no category is detected
4. Added regression test: `test_structured_order_query_uses_count_by_category`
5. Re-ran the full test suite — 232 passing

DECISION POINT — BOOLEAN VALIDATOR GAP

During code review of `analytics_tools.py`, I found that Claude's `_require_nonempty_str()` validator accepted `True` as a valid string input. Python's `bool` is a subclass of `int`, so a naive `isinstance(x, str)` check would correctly reject `True` — but the companion `_require_positive_int()` validator would accept `True` (since `True == 1`). Added explicit `isinstance(x, bool)` rejection before the int check.

DECISION POINT — OBSERVATION STORAGE BOUNDARY

During the memory layer review, I noticed that `update_session_from_state()` was reading from the observation field to populate session metadata. This would cause raw dataset rows to be written to disk. I revised the function to extract metadata exclusively from `tool_input` — the observation is ephemeral and must never reach persistent storage.

Other significant refinements

- **Removed duplicated validation:** MCP server initially had inline validators inside tool functions that duplicated Pydantic schema checks — removed to eliminate inconsistency risk.
- **Added `read_only=True` to DuckDB connections:** Analytic consumers should never mutate the database. Initial Claude code opened read-write connections.
- **Graceful JSON fallback:** Session restore from corrupted files raised unhandled `JSONDecodeError`. Added `try/except` with fresh session creation on failure.

- **Chain-of-thought boundary:** Verified that the trace list (internal LLM steps) is never passed to `_print_trace()`. Only operational fields are printed.
-

FUNDAMENTAL PRINCIPLE

Claude Code accelerated implementation significantly, but its output required engineering supervision at every step. I treated AI-generated code as a **draft**, not a source of truth. Every layer was reviewed through `git diff`, `pytest`, manual CLI validation, and architecture comparison checkpoints.

Where AI excelled

- **Scaffolding:** producing correct boilerplate structure for LangGraph graphs, Pydantic models, FastMCP wrappers, and pytest fixtures at speed
- **Pattern application:** consistently applying parameterized query patterns once the requirement was stated clearly
- **Test generation:** producing comprehensive adversarial and failure-mode test cases that I would have taken much longer to enumerate manually
- **Documentation:** generating structured evaluation plans, security audit documents, and architecture notes from specification prompts

Where AI required correction

- **Routing logic subtleties:** the tool-selection bug (ORDER query → wrong tool) was not caught by AI-generated tests. Only manual CLI validation surfaced it.
- **Boundary conditions in validators:** the boolean subtype issue in `_require_positive_int()` was not caught by the initial generation.
- **Over-validation:** Claude added redundant inline validation inside MCP tool functions that duplicated Pydantic schema logic — creating maintenance risk rather than safety.
- **Memory boundary:** Claude initially stored observations in session state. This required explicit correction to enforce the "never store observations" rule.

Working style with Claude Code

The most effective pattern was: *constrained prompt* → *implementation* → *review* → *targeted correction* → *regression test* → *commit*. Broad open-ended prompts produced more hallucination surface. Narrow, rule-heavy prompts (like "parameterized queries only, no string concatenation") produced more predictable and correct output.

The CLAUDE.md files — one per layer — were essential. They kept Claude's context anchored to the architectural constraints even across sessions. Without them, Claude would occasionally drift toward shortcuts (e.g., f-string SQL building) that violated the system's core invariants.

07 Testing Approach

FINAL RESULT

232 passing tests. All deterministic. No external API calls, no network dependencies. All use isolated

`tmp_path` DuckDB fixtures with known data (3–5 rows). Reproducible on any machine.

Testing strategy overview

TEST FILE	TYPE	COUNT	WHAT IT COVERS
<code>test_router.py</code>	Unit	13	Each route type, confidence ranges, OOS priority, no chain-of-thought in route result
<code>test_tools.py</code>	Unit	45	All 5 analytics tools, error handling, case-insensitivity, type validation, latency logging
<code>test_agent_execution.py</code>	Integration	17	End-to-end flows through LangGraph, trace contract enforcement, regression cases from bugs found manually
<code>test_mcp_server.py</code>	Unit + Integration	23	Pydantic schema validation, MCP wrapper delegation, rejection of invalid inputs
<code>test_memory.py</code>	Unit	28	Session/profile persistence, corrupted file recovery, NEVER_STORE enforcement, path traversal sanitization
<code>test_cli.py</code>	Unit	19	EXIT_COMMANDS behavior, trace field presence, no chain-of-thought exposure, session persistence per turn
<code>test_adversarial_queries.py</code>	Adversarial	30	SQL injection payloads, prompt injection, Unicode RTL override, null bytes, control chars, jailbreak attempts
<code>test_failure_modes.py</code>	Failure	24	Corrupted DuckDB, missing files, empty dataset, extreme-length inputs, concurrent access patterns
<code>test_security_edges.py</code>	Security	27	Data leakage prevention, session ID path traversal, observation boundary, chain-of-thought boundary

How the test types fit together

Unit tests verify that each component is correct in isolation. Integration tests verify that components wire together correctly through the full LangGraph graph. Regression tests pin specific bugs found during manual validation so they cannot re-emerge silently. Security and adversarial tests verify that the system's trust boundaries hold even under hostile inputs. Failure mode tests verify graceful degradation when infrastructure assumptions fail.

Example: regression test from the ORDER routing bug

```
def test_structured_order_query_uses_count_by_category(tmp_path):
    # Regression: "How many ORDER requests?" was routing to count_total_rows
    db = setup_test_db(tmp_path)
    result = run_graph("How many ORDER requests?", db_path=db)
    assert result["tool"] == "count_by_category"
    assert result["tool_input"] == "ORDER"
    assert result["observation"] != 26872 # must not be total row count
```

08

What I Chose Not to Test

Choosing what not to test is as important as choosing what to test. The following were intentionally excluded, with explicit reasoning for each.

WHAT WAS EXCLUDED	WHY EXCLUDED	RISK ACCEPTED
Concurrent multi-user execution	DuckDB's <code>read_only</code> mode makes this safe by design; concurrent writers require a different deployment model	Untested under high concurrency — no connection pooling
Real external LLM judging	Introduces API dependency, cost, and non-determinism; evaluation plan documents the rubric and scoring instead	LLM-as-judge quality is advisory, not automated
Distributed MCP deployment	FastMCP is tested locally; production network deployment requires infrastructure outside this scope	Network-level auth and TLS not verified
Production authentication	No auth layer was designed — session ID is informational, not a security boundary	Any caller can impersonate any session ID
Log rotation and atomic writes	Python's <code>logging</code> module was sufficient for the submission scope; production would use structured log aggregation	Log file can grow unbounded; writes are not atomic
Real semantic RAG retrieval	RAG was scoped as future work; including a vector store would add external dependencies and undermine determinism	Exploratory queries use extractive examples only

BEHAVIORS THAT REMAIN UNVERIFIED

The system has not been tested with: more than one concurrent user session, a network-accessible MCP deployment, a production LDAP/OAuth authentication layer, or a real LLM judge scoring 33 golden dataset entries. These represent the gap between this submission and a production deployment.

A BUG THAT SLIPPED PAST TESTS

The ORDER routing bug (Section 05) was not caught by any AI-generated test. The generated tests for the structured node verified that the route was classified as "structured" — but did not assert which specific tool was selected or what the tool input was. This gap was closed after manual CLI validation exposed the bug. The lesson: tool-selection assertions must be first-class test requirements, not afterthoughts.

09 Security & Governance

The system exposes operational traces (route, selected tool, tool input, observation, final answer) but **never exposes hidden chain-of-thought**. The following boundaries are enforced by design and verified by test.

SQL Injection Prevention

- Parameterized DuckDB queries — no string concatenation
- `contains()` function treats wildcards as literals
- Input validated before any SQL is constructed
- Verified: `' ; DROP TABLE--`, `UNION SELECT` return zero rows, no error

Memory Leakage Prevention

- Session reads only `tool_input` metadata — never observation
- Profile blocked by `_NEVER_STORE` frozenset: `api_key`, `token`, `password`, `secret`, `credential`, `email`, `phone`, `ssn`, `raw_query`, `dataset_row`
- 500-character limit on stored query strings
- Defensive strip on save

Session ID Safety

- Sanitized with `re.sub(r'^[a-zA-Z0-9_\-]', '', ...)`
- 64-character cap
- Path traversal sequences (`../`, `..\`) stripped
- Verified: `../ ../ ../etc/passwd` → `etcpasswd`

Chain-of-Thought Boundary

- Internal trace list never passed to `_print_trace()`
- CLI prints only: route, confidence, tool, truncated observation, final answer
- No intermediate LLM reasoning steps exposed
- Verified by `test_security_edges.py`

Evaluation governance

The `docs/evaluation_plan.md` defines a 33-entry golden dataset across 6 route categories (STRUCTURED, UNSTRUCTURED, OUT_OF_SCOPE, CLARIFICATION, ADVERSARIAL, MALFORMED). Scoring rubric: 1–5 scale, minimum pass threshold of 3. LLM-as-judge is explicitly constrained: it may never override DuckDB counts and its output is advisory only. Target: 90% pass rate per route category.

CLAUDE.MD AS GOVERNANCE MECHANISM

Each module layer has its own `CLAUDE.md` file with binding rules for that layer. This means that even when working across sessions, the AI assistant operates within the same architectural constraints. It is a governance mechanism, not just documentation.

10 Demo & Examples

The demo covers the four route types and the full test suite. Key interactions:

Structured analytical query

```
user > How many records are in the dataset? — Trace — route
structured confidence 0.90 tool count_total_rows tool_input (none) observation 26872
answer The dataset contains 26,872 customer service
records.
```

Category-specific count query

```
user > How many ORDER requests are in the dataset? — Trace — route
structured confidence 0.90 tool count_by_category tool_input ORDER observation 3,245
answer There are 3,245 ORDER-related requests in the
dataset.
```

Unstructured query

```
user > Show me examples about refund — Trace — route
unstructured confidence 0.88 tool search_examples tool_input refund observation [5 dataset rows
returned – truncated in trace]
answer Here are
representative examples related to 'refund': [formatted examples]
```

Out-of-scope refusal

```
user > Who won the Champions League? — Trace — route
out_of_scope confidence 0.95 tool (none)
answer This
question is outside the scope of the customer service dataset. I can only answer questions about
customer support interactions, categories, and intents.
```

Full test suite

```
$ pytest tests/ -v --tb=short
```

```
test_router.py ..... PASSED (13 tests)
test_tools.py ..... PASSED (45 tests)
test_agent_execution.py ..... PASSED (17 tests)
test_mcp_server.py ..... PASSED (23 tests)
test_memory.py ..... PASSED (28 tests)
test_cli.py ..... PASSED (19 tests)
test_adversarial_queries.py ..... PASSED (30 tests)
test_failure_modes.py ..... PASSED (24 tests)
test_security_edges.py ..... PASSED (27 tests)
```

```
===== 232 passed in 4.81s =====
```

Smart Hybrid RAG — the right boundary

DESIGN PRINCIPLE FOR RAG EXTENSION

Structured analytics remain deterministic. Semantic RAG would complement the system for exploratory questions only — where exact counts are not required and the user wants semantically similar examples rather than statistical distributions.

If RAG were added, each dataset row would become one document with `instruction + response` as text and `category, intent, flags` as metadata. The router would add a fifth route: `semantic`. The critical invariant: the RAG path would never be used for counting, filtering, or distributional queries. Those remain deterministic DuckDB operations forever.

FUTURE ENHANCEMENT	RATIONALE	CONSTRAINT
Semantic RAG (vector store)	Exploratory "find similar" queries benefit from embedding similarity	Must not replace DuckDB for counts/distributions
Production authentication	Session IDs must be tied to real identity for multi-tenant use	OAuth2 / LDAP integration required
Real LLM-as-judge evaluation	Automate the 33-entry golden dataset scoring	DuckDB counts must never be overridable by the judge
Streaming responses	Long unstructured summaries benefit from streaming tokens	Trace contract must still be complete before stream ends
Log aggregation	Production requires structured logging (JSON, Datadog, etc.)	No sensitive data in log fields
Multi-dataset support	Generalize beyond Bixtext to other customer support datasets	Schema validation required per dataset at ingest time

Conclusion

This project demonstrates what a production-oriented AI analytics agent looks like when built with **engineering discipline** rather than just LLM convenience. The core insight is that LLMs and deterministic analytics are complementary, not competing — but they must be kept in separate, clearly bounded layers.

What this project demonstrates

- Clear problem framing for AI systems
- Constrained, scoped prompting for Claude Code
- Critical review of AI-generated code before accepting
- Systematic testing across nominal, adversarial, and failure paths
- Security-first design with explicit trust boundaries
- Governance through CLAUDE.md layer rules

Core invariants that hold

- All numerical answers come from DuckDB — not LLM generation
- No raw user input concatenated into SQL
- No observations stored in persistent memory
- No chain-of-thought exposed to CLI
- Out-of-scope queries refused, not deflected
- 232 tests enforce these invariants continuously

FINAL NOTE ON AI-ASSISTED DEVELOPMENT

The right relationship with an AI coding assistant is: *trust its scaffolding, verify its logic, own its boundaries*. Claude Code enabled a 5-layer production architecture with comprehensive test coverage at a pace that would be significantly slower manually. But every boundary — SQL parameterization, observation isolation, chain-of-thought suppression — required human review to get right. AI accelerates; engineering judgment ships.